

Predicting College Basketball Outcomes: A Win Probability Model for the Atlantic 10

Samantha Taylormoore, Blake Craig, Adriana Risi

Dr. Tim Chartier

MAT/CSC 210

6 May 2025

Abstract

In this project, we developed a win probability model to predict outcomes of college basketball games within the Atlantic 10 conference. Focusing on Davidson and its opponents, we used historical and current-season game data to create a weighted scoring system based on effective field goal percentage, turnover percentage, defensive rebounding percentage, and free throw rate. Our model aggregates team statistics up to the matchup date, accounts for opponent strength, and normalizes scores to produce a pre-game win probability. Using custom Python scripts for data cleaning, feature engineering, and GUI development, we evaluated the model's accuracy across two seasons of data. Our results demonstrate that our model can offer valuable predictive insight, and we outline potential future improvements, including incorporating player-level and in-game statistics for dynamic prediction.¹

Problem description

The goal of this project is to develop a model that predicts the outcome of Atlantic 10 college basketball games by calculating pre-game win probabilities based on team-level performance metrics. The central question we set out to answer was whether we could accurately determine the probability that one team would defeat another based on their statistical performance leading up to a specific matchup. To pursue this question, we gathered advanced game-level statistics, implemented a standardized dataset spanning two seasons, and designed a model that aggregates these statistics into a single pre-game win probability. We also examined which metrics most effectively contributed to predictive accuracy and evaluated how closely our

¹Full implementation and source code are available at: github.com/samtaylormooredm/Win-Probability-CSC210.

custom weightings aligned with established basketball analytics frameworks. Through this modeling process, we also explored the model's limitations, such as how the absence of player-level data or game context might affect accuracy. Ultimately, our goal was to create a model that offers insight into game outcomes and highlights the predictive power of key team statistics.

Simplifications

To make the project feasible within time and data constraints, we made several simplifications. First, we based the model on game-level statistics, where each data point represents a full game rather than possessions, quarters, or time intervals. This choice streamlined data collection and processing, but prevented us from modeling the dynamic shifts that occur during a game, such as momentum swings or late-game strategy.

Second, we focused exclusively on team-level metrics rather than incorporating player-specific data like individual scoring rates, playing time, or injuries. While this simplification allowed us to standardize inputs across teams and seasons, it limits the model's ability to capture performance fluctuations caused by roster changes or player availability.

Finally, while the model accounts for prior games leading up to a matchup, it assumes all historical games contribute equally within a season and does not dynamically adjust for trends such as recent winning or losing streaks. In future work, integrating possession-level data, player statistics, and temporal trends could enhance the model's realism and predictive accuracy.

Mathematical model

A significant portion of setting up our mathematical model was dedicated to cleaning and organizing the data. We began by collecting advanced game log data from Sports-Reference.com, focusing on A-10 game results from the 2023-2024 and 2024-2025 seasons. For each opponent, we downloaded the full Excel Workbook version of their advanced game logs. To make the data easier to process with Python, we wrote a script using pandas to convert each Excel file into a standardized CSV format. Each file was parsed, the relevant data was extracted, and the output was saved to an additional data folder organized by season. Throughout this process, we used ChatGPT to help generate and refine portions of the data

processing code, especially when automating repetitive tasks or debugging complex pandas operations.

Once in CSV form, we ran each file through a second custom script that automated the cleaning process. This included removing irrelevant columns and renaming inconsistent columns, particularly those with ambiguous or duplicate names across teams. We also added new metadata columns, such as *Team*, indicating which team the season data belonged to; *Season*, specifying the season in which each game occurred; and *Source_File*, tracking the original file source for each game log. All cleaned DataFrames were appended to a list and concatenated into a single dataset with nearly 2,000 rows of game-by-game data. This consolidated dataset served as the basis for our modeling process. A sample of the final dataset is shown in Figure 1.

Date	Season	Team	Opp	Reslt	Points	Opp_Points	eFG%	TOV	TOV%	ORB	ORB%	FT%	STL	BLK	FGA	FTA	Opp_eFG%	Opp_TOV%	Opp_TOV%	Opp_ORB%	Opp_ORB%	Opp_FT%	Opp_STL	Opp_BLK	Opp_FGA	Opp_FTA
2023-11-06	2023-2024	Arizona	Morgan State	W	122	59	0.63	7	0.07	20	0.54	0.34	16	7	76	32	0.46	25	0.29	6	0.18	0.14	3	6	56	15
2023-11-10	2023-2024	Arizona	Duke	W	78	73	0.51	17	0.19	12	0.35	0.18	8	5	65	18	0.49	11	0.14	5	0.15	0.14	9	3	65	12
2023-11-13	2023-2024	Arizona	Southern	W	97	59	0.6	23	0.24	13	0.43	0.48	7	2	58	36	0.36	14	0.16	6	0.15	0.21	12	2	63	19
2023-11-17	2023-2024	Arizona	Belmont	W	100	68	0.6	14	0.15	16	0.48	0.26	7	4	66	27	0.48	14	0.17	4	0.11	0.16	9	4	61	17
2023-11-19	2023-2024	Arizona	UT Arlington	W	101	56	0.6	11	0.12	10	0.34	0.19	12	7	73	15	0.42	18	0.21	5	0.14	0.08	6	5	61	12
2023-11-23	2023-2024	Arizona	Michigan State	W	74	68	0.5	10	0.13	9	0.28	0.25	8	5	59	20	0.44	9	0.11	9	0.29	0.17	6	2	64	14

Figure 1: Sample of the cleaned dataset with rounded data entries.

Solution of the mathematical problem

After assembling the cleaned dataset, we turned to identifying the specific variables that would most effectively drive the win probability model. Our goal was to focus on game-level metrics that would offer insight into both performance and outcomes for Davidson and their opponents. Our initial framework was inspired by Dean Oliver's *Four Factors of Basketball Success*, which established the following recommended weights:

- Effective field goal percentage: (eFG%) – 40% weight
- Turnover percentage (TOV%) – 25% weight
- Offensive rebounding percentage (ORB%) – 20% weight
- Free throw rate (FT%) – 15% weight

(Oliver).

Rather than adopting these percentages exactly, we adjusted the weights used in our model to better fit the A-10 conference data we collected. We used ChatGPT to help create and refine our `generate_weights.py` script, which applied linear regression to train a customized weighting system that adjusted the relative importance of each factor based on the actual game outcomes in the cleaned dataset. In addition to the original *Four Factors*, we expanded the feature set slightly by introducing steal rate and block rate. Although these defensive metrics received low weights of 0.5% each, they nonetheless improved the model's predictive accuracy by capturing defensive disruptions more fully.

The final weights assigned to each factor were:

- Effective field goal percentage (eFG%) – 41% weight
- Turnover percentage (TOV%) – 43% weight
- Offensive rebounding percentage (ORB%) – 4% weight
- Free throw rate (FT%) – 11% weight
- Steal rate (% of opponent possessions ending in steals) – 0.5% weight
- Block rate (% of opponent possessions ending in blocks) – 0.5% weight

Notably, the data-driven weights we found place a greater emphasis on turnover percentage compared to Oliver's original 2003 framework. In particular, we also found that our findings align with more recent modifications of the *Four Factors*, such as those published by Statathlon, which suggest relative weightings of 43% for eFG, 39% for TOV, 10% for ORB, and 8% for FT (Kotzias). This validation gave us confidence that our model's weighting system was both statistically grounded and consistent with modern basketball analytics trends.

Each team's performance was first adjusted to account for their opponent's defensive strengths. Specifically, for each statistic, we subtracted the opponent's defensive average from the team's offensive average to calculate a "net advantage." For example, a team's Net eFG% is found by taking its own eFG% and subtracting the opponent's allowed eFG%. This method highlights the matchup-specific differences between teams, rather than relying on absolute performance metrics alone. After calculating the net advantages, we applied a weighted sum to each team's metrics to produce a raw score. These raw scores were then normalized by dividing each team's score by the combined total of both teams' scores. To ensure valid probability values, any negative raw scores were first floored to zero before normalization.

To structure the solution programmatically, we developed a Python script, `a10_predictor.py`, which incorporates data processing, prediction modeling, and an interactive GUI. Throughout development, we also used ChatGPT to assist with refining logic, debugging errors, and streamlining parts of the code. Below is the pseudocode summary of the key components:

Python

`a10_predictor.py` pseudocode:

1. LOAD Data
 - Load basketball game data **from** a CSV file.
 - Parse dates into datetime objects.
2. DEFINE FUNCTION `extract_team_list(data)`
 - Extract unique teams **from** the dataset.
 - Filter the teams to include only A-10 conference members.
3. DEFINE FUNCTION `get_pre_matchup_games(team_name, opponent_name, season, all_data)`
 - Filter `all_data` **for** games played by `team_name` during the specified season.
 - Sort the games by date.
 - Return **all** available games **for** that team **in** the season.
4. DEFINE FUNCTION `compute_advanced_stats(past_games)`
 - Calculate average values **for** the following statistics:
eFG%, TOV, TOV%, ORB, ORB%, FT%, Opp_eFG%, Opp_TOV, Opp_TOV%, Opp_ORB%, Opp_FT%, FGA, FTA, Opp_FGA, Opp_FTA, STL, BLK
 - Estimate opponent possessions using:
 $\text{Opponent Possessions} = \text{Opp_FGA} + 0.44 \times \text{Opp_FTA} + \text{Opp_TOV}$
 - Compute:
 - Steal rate = $(\text{STL} / \text{opponent possessions}) \times 100$
 - Block rate = $(\text{BLK} / \text{opponent possessions}) \times 100$
 - Calculate net advantage metrics:
 - Net_TOV%, Net_eFG%, Net_FT%, Net_ORB%, Steal%, Block%
 - Return a dictionary of computed advanced stats.
5. DEFINE FUNCTION `predict_game_winner(team1, team2, season, data)`
 - Get past games **for** `team1` and `team2` using `get_pre_matchup_games()`.
 - Compute each team's advanced stats using `compute_advanced_stats()`.
 - Calculate raw scores **for** each team:
 $\text{Score} = \text{sum}(\text{weight} \times \text{stat_value} \text{ for each stat in WEIGHTS dictionary})$
 - Normalize scores into win probabilities:
 $\text{Win probability} = \text{team_score} / (\text{team_score} + \text{opponent_score})$
 - Return the predicted winner and the win probability.
6. DEFINE FUNCTION `run_gui()`
 - Build a Tkinter window titled "A-10 Basketball Game Predictor".
 - Add two dropdown menus to select Team 1 and Team 2.
 - Add a "Predict Winner" button to run the prediction.
 - Add a "Reset" button to clear selections.

```

- On clicking "Predict Winner":
  - Read the selected teams.
  - Call predict_game_winner().
  - Display the predicted winner and win probability.
  - Show error messages for invalid inputs.

7. DEFINE FUNCTION test_model_accuracy(all_data, season="2024-2025")
  - Loop through all games in the given season.
  - For each game:
    - Predict the winner using predict_game_winner().
    - Compare the predicted winner to the actual result.
    - Track the number of correct predictions and total predictions.
  - Calculate and print model accuracy as correct / total.

8. MAIN Execution Block
  - Print "Testing model accuracy (A-10 teams only)..."
  - Call test_model_accuracy() to evaluate accuracy.
  - Launch the GUI by calling run_gui().

```

Results

After constructing the underlying model logic, we developed an interactive GUI to visualize and interact with the win probability predictions. Running the completed `a10_predictor.py` program launches the GUI, allowing users to dynamically select two A-10 teams, predict the likely winner based on historical performance metrics, and immediately view the calculated win probability. A screenshot of the predictor in use is shown in Figure 2.

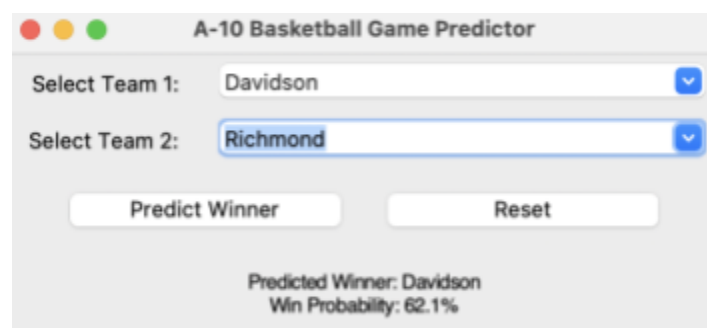


Figure 2: GUI Screen for Davidson vs Richmond win prediction.

In addition to building the GUI, we implemented an accuracy testing function within the script to evaluate the model's predictive performance. The `test_model_accuracy()` function uses each game's actual matchup date to simulate realistic prediction conditions. For each 2024–2025 season game, it predicts the outcome by only considering past games played before

that date. This avoids any future data leakage and ensures that the model's predictions reflect the information that would have been available at the time of the game.

Upon running the accuracy test, the model achieved an overall prediction accuracy of 73.33% for the 2024-2025 season, correctly predicting 110 out of 150 total games. This result indicates that the model effectively captures important statistical patterns that influence game outcomes, even when restricted to information available prior to each matchup. Since our model was developed based on data from only a couple of seasons within the A-10 conference, a valuable future step would be to expand the evaluation across additional conferences and seasons. Testing the model on broader datasets could reveal how well it generalizes beyond the A-10 and whether it can better capture patterns of randomness and variability present in a larger set of teams and playing styles.

Improvements

While our current model performs well with a 73.33% accuracy rate, there are several improvements that could yield even better predictive results. One major enhancement would be the incorporation of player-level data, such as individual scoring efficiency, playing time, and injury status, since our model relies on team-level averages, limiting and minimizing the ability to predict the real-time impact. Including player data could better account for how lineup changes or the absence of key players influence outcomes. Implementing this would be a major increase in data required, such as access to reliable, regularly updated player statistics, which is a possible next step with the right data. Additionally, expanding our dataset beyond two seasons and beyond the A-10 conference could allow the model to learn from a wider variety of team strategies and performance patterns, however wasn't feasible for our project timeline since we had to manually download each team's data for each season.

A second significant improvement would be to incorporate temporal trends. As it stands, the model weighs all games within a season equally, which overlooks important nuances such as recent winning or losing streaks, mid-season performance shifts, rivalries, or home vs. away dynamics. Adjusting the model to emphasize certain games, track team momentum, or factor in schedules such as travel and rest days could increase its accuracy. This would require some algorithmic changes but could substantially boost prediction accuracy. Future models might also explore in-game prediction capabilities by including possession statistics, allowing the model to

update win probabilities live as a game occurs. Although more challenging to implement due to the need for finer data and real-time processing, this improvement would bring the model closer to practical applications in coaching, broadcasting, and fan engagement.

Conclusions

Ultimately, our project demonstrates that a data-driven model based on team-level performance metrics can serve as an effective foundation for building reliable sports analytics tools. By customizing the weighting of key variables and accounting for opponent strength through net statistical advantages, our model offers a structured approach to forecasting game results based on pre-game data alone. Our work also illustrates how simplifications, such as omitting player-level data and temporal trends, can make a project more manageable while still yielding meaningful results. At the same time, these simplifications expose areas for future improvement. Expanding the model to include dynamic performance trends, individual player statistics, and more diverse datasets could significantly increase accuracy and generalizability. Overall, this project highlights the predictive power of well-selected team metrics and the value of aligning model design with both statistical rigor and practical constraints. With further refinement and additional data, the framework we built could serve as a foundation for more advanced sports analytics tools.

Acknowledgements

We would like to thank Dr. Tim Chartier for his guidance and support with this assignment, as well as 'Cats Stats for their resources and recommendations in sourcing relevant data to support this project.

Works Cited

“Basketball Reference Four Factors.” *Sports Reference*,

www.basketball-reference.com/about/factors.html. Accessed 28 Apr. 2025.

Chatgpt, chatgpt.com/. Accessed 28 Apr. 2025.

Kotzias, Konstantinos. “The Four Factors of Basketball as a Measure of Success.” *Statathlon*, 17

Sept. 2023, statathlon.com/four-factors-basketball-success/.

“Sports Reference: Sports Stats, Fast, Easy, and up-to-Date.” *Sports Reference*,

www.sports-reference.com/. Accessed 28 Apr. 2025.